

Script Presentatore — Demo "Rome Future Week" (Crm-A Console)

Percorso consolidato della demo, beat per beat. Durata indicativa: **~25–28 min.**
Tono: vendita/educazione, ritmo **"memoria → campagna → chiamata vera → riconoscimento"**.

Niente CURL sul palco. Ogni azione è una di queste tre cose:

1. un **comando in chat** all'agente (Copilot);
2. un **click in console** (profilo, scheda campagna, dashboard NLPearl);
3. una **chiamata telefonica vera** (NLPearl fa l'AI della conversazione, la Console è il cervello CRM: riconosce, registra, orchestra).

I **curl** esistono solo come verifica tecnica **in preparazione** (sezione "Checklist pre-demo"), mai durante la demo.

Premessa narrativa (era l'Atto 6 dello script precedente — ora è il punto di partenza): **Lorenzo è già cliente.** Una settimana fa ha comprato il Galaxy S26. Oggi il corriere GLS è in carico: consegna prevista domani entro le 18. Il CRM lo sa. Da qui parte tutto.

0. Apertura & setup (3 min) — "Il CRM che già ricorda"

 **Scena:** schermo condiviso, console aperta sull'URL pubblico (funnel Tailscale, es. <https://crm-a-console.<tailnet>.ts.net>), chat Copilot pronta. Telefono del presentatore (= numero di Lorenzo, vedi prep) in vivavoce. Dashboard NLPearl in un secondo tab.

 **Azione presentatore (solo prep, NON mostrata):**

- Seed eseguito; profilo **Lorenzo** aggiornato: Phone = tuo numero, Canale preferito = **Telefono**, Opt-in marketing = sì, ordine S26 in consegna (GLS).

- Pearl **inbound** creata e **attiva** (dal prompt `NLPEARL-SERVICES-PROMPT.md`), numero inbound pronto a squillare.
- Pearl outbound di collaudo **rimossa** dalla dashboard.

Mostra in console:

1. Profilo Lorenzo: anagrafica, consenso, **ultimo ordine Galaxy S26 — GLS, "in carico oggi, consegna domani entro le 18"**, timeline con la chiamata di una settimana fa.
2. **Integrations** → **card NLPearl** (credenziali + URL webhook) e **Skills** → **nlpearl**.

Script:

"Buongiorno. Crm-A Console è un CRM-AI interamente locale: non un database di contatti, ma un *cervello* che parla con i vostri sistemi di comunicazione — telefono, email, Telegram. E il bello: è già un cliente che abbiamo. Lorenzo ha comprato il Galaxy S26 la settimana scorsa — il corriere è in carico, consegna domani entro le 18. La Console lo sa: lo vedete nella scheda. Questo è il punto di partenza di tutto: **la memoria c'è già, prima che suoni una chiamata.**"

(Mostra card NLPearl e skill nlpearl.) "Questo è il telefono della Console: NLPearl fa l'AI delle conversazioni, e la Console — raggiungibile a questo URL pubblico — risponde con identità, storico e acquisti. Da qui NLPearl ci richiama a ogni evento."

(Mostra l'URL pubblico.) "Niente server da gestire: il tunnel espone la Console, e le chiamate che vedrete squillare sono vere."

 **Pubblico vede:** profilo Lorenzo con ordine S26 + timeline; card NLPearl; skill nlpearl; URL pubblico `...ts.net`.

★ Atto 1 — La campagna nasce dalla chat (5 min) *(il motore della demo)*

Obiettivo: mostrare il tool `crm_a_phone_campaign`: l'agente, da solo, crea la scheda, pilota NLPearl e chiede conferma prima di ogni invio. Da qui nasce la chiamata vera dell'Atto 2.

 **Azione:** in chat, digitare:

"Crea una campagna outbound per il Galaxy 27, con una breve comparazione col modello precedente (S26), usando il numero outbound 686fd112a91849a9e59a5353, e chiama chi preferisce il telefono."

Script (beat):

(L'agente chiama il tool con *upsert*.) "Guardate: l'agente crea la **scheda campagna** — prodotto, confronto col modello precedente, config telefonica: finestra 9–18, lun–ven, fuso di Roma, tre tentativi massimi, un agente. E non vi mostra codice: lo fa lui, dietro le quinte."

(L'agente chiama *create*.) "Ora crea la **Pearl su NLPearl** — l'agente vocale della campagna. Nota: parte **in pausa**. Nessuna chiamata finché non la attivo io. Il sistema non scatta da solo."

(L'agente mostra l'anteprima audience e chiede conferma per *send* — mai in automatico.) "Ora definisce *chi chiamare*: solo chi ha dato il consenso telefonico — nel nostro caso Lorenzo. E **chiede a me la conferma prima di mandare i lead**."

(Confermare, poi mostrare l'output *leadsCreated: 1*.) "Ecco il lead enqueued. La campagna è pronta, ancora in pausa. La attivo solo quando confermo io."

👁️ **Pubblico vede:** i tool-call in chat (*upsert* → *create* → *send*); la scheda campagna con Voice Brief; Pearl *Paused* nella dashboard NLPearl; il gate di conferma; *leadsCreated: 1*.

Regole sul palco:

- *send/resume* **solo** dietro conferma esplicita (*confirm: true*).
- Il numero outbound nella chat va sostituito con quello **verificato** per l'account (vedi Note NLPearl nel runbook); se l'agente lo scopre da solo, ometterlo.

Atto 2 — La prima chiamata vera: memoria + campagna in una sola chiamata (5 min)

Obiettivo: la campagna chiama **davvero**; il CRM riconosce il numero, registra l'esito e il consenso.

 **Azione:** in chat, confermare `resume` (o `send + resume`) → il Pearl NLPearl inizia a chiamare → **squilla il telefono del presentatore** (è il numero di Lorenzo). Telefono in vivavoce: il pubblico sente l'AI. Rispondere seguendo il copione.

SCRIPT CHIAMATA OUTBOUND (Pearl campagna → Lorenzo):

[AI] "Buongiorno Lorenzo, una chiamata per conto di Crm-A Console. Vorremmo presentarle una nuova offerta." **[Lorenzo]** "Buongiorno. Sì, mi interessa — vi ho comprato il Galaxy S26, che offerta c'è per me?" **[AI]** *(dal Voice Brief: caratteristiche S27, differenze vs S26, promo)* "Il nuovo Galaxy S27 arriva il 18 ottobre: chip più veloce, batteria maggiore, fotocamera molto migliorata in bassa luce, sette anni di aggiornamenti. E per i possessori del Galaxy S26 c'è la promo di lancio: **fino a 150 euro di bonus permuta.**" **[Lorenzo]** "Mi interessa. Ma potete ricontattarmi? E datemi pure la promo." **[AI]** *(raccoglie il consenso esplicitamente)* "Certamente. Confermo il suo consenso a ricevere comunicazioni di marketing e ad essere ricontattato per il lancio. Preferisce il telefono per le prossime comunicazioni?" **[Lorenzo]** "Sì, il telefono va bene." **[AI]** "Perfetto. La ricontatteremo al lancio. Grazie Lorenzo, buona giornata."

(L'AI improvvisa dal Voice Brief: queste sono le battute attese, non un copione cablato.)

Dopo la chiamata — mostrare in sequenza:

1. **Dashboard NLPearl:** chiamata completata, registrazione + trascrizione + summary.
2. **Console → profilo Lorenzo:** la **timeline** mostra la nuova interazione `Call` — il webhook ha riconosciuto il numero e l'ha agganciata alla persona giusta (nessun doppione, nessun contatto fantasma).
3. **Scheda campagna:** lo stato del lead (`campaign_send`) è passato a **Success**.
4. Poi in chat:

"Consolida i dati della chiamata: consenso marketing sì, canale preferito telefono."

(L'agente aggiorna il profilo. Il consenso non viene scritto in automatico: l'operatore conferma. Altro gate umano.)

Script:

"Il telefono squilla davvero — è la Pearl della campagna che chiama. E guardate cosa succede qui: la Console riconosce il numero, lo aggancia a Lorenzo — non

è un estraneo, è un cliente — e a fine chiamata registra l'esito in timeline. Poi io chiedo all'agente di consolidare il consenso raccolto dalla chiamata: marketing sì, canale preferito telefono. Un contatto pieno, aggiornato, con la sua storia."

👁️ **Pubblico vede:** telefono che squilla + voce AI; call record su NLPearl; interazione `Call` in timeline; `campaign_send Status = Success`; profilo aggiornato dopo il comando di consolidamento.

Atto 3 — Il CRM diventa memoria (Copilot) (2 min)

💻 **Azione:** nella chat, digitare:

"Stiamo lanciando il nuovo Samsung Galaxy. Qual è il pubblico migliore a cui proporlo?"

🎙️ **Script:**

"Ora il CRM ragiona. Niente query scritte da un DBA: l'agente usa la sua skill CRM, legge il segmento di lancio, incrocia chi ha opt-in e le preferenze di canale, e risponde. Ecco Lorenzo in lista — cliente S26, consenso sì, canale telefono. Questo è il potere di una base dati *viva*."

👁️ **Pubblico vede:** risposta dell'agente con la lista del pubblico e il ragionamento su canali e consensi.

Atto 4 — Orchestrazione multicanale (4 min)

💻 **Azione:** nella chat, digitare:

"Prepara il brief di lancio del Galaxy 27, con il confronto col modello precedente, e invia l'annuncio al segmento 'Lancio Samsung Galaxy' per canale preferito."

🎙️ **Script (beat):**

(L'agente scrive il file `brief-galaxy-s27.md` nel workspace e lo apre.) "Il prodotto 'viaggia' come un brief di marketing: caratteristiche, differenze col S26, promo lancio, link d'acquisto. È la stessa conoscenza che ha parlato al telefono nell'Atto 2 — e ora serve anche ai canali scritti."

(L'agente lancia l'invio multicanale e mostra il risultato.) "Stessa campagna, instradata **per canale preferito**: chi ha scelto Telegram la riceve su Telegram, chi email su email. Ecco il risultato: `sent`, i destinatari per canale, `failed: []`."

👁 **Pubblico vede:** file `brief-galaxy-s27.md` aperto; JSON `{ ok, sent, telegram:[...], email:[...], failed:[] }`.

Nota onesta sul palco: l'invio reale su Telegram/email richiede i canali collegati. Se in prova qualche canale fallisce, narrarlo senza nascondere: "i canali live della demo sono il telefono — l'avete appena visto; qui vedete l'instradamento per canale". Collaudare prima.

Atto 5 — Il cliente richiama: la memoria risponde (5 min)

Obiettivo: il momento "wow": Lorenzo richiama e la Console lo riconosce — nome, ordine in consegna, e la nuova offerta.

📞 **Azione:** il presentatore, dallo stesso telefono, **chiama il numero inbound** della Console (Pearl inbound attiva). Telefono in vivavoce. Prima di comporre, narrare; poi seguire il copione; a fine chiamata commentare.

📞 **SCRIPT CHIAMATA INBOUND (Lorenzo → Pearl inbound):**

[AI] "Buongiorno Lorenzo, come posso aiutarla?" [Lorenzo] "Buongiorno. Sì, è Crm-A? Volevo sapere del mio Galaxy S26 — dove è la consegna?" [AI] *(dal contesto CRM restituito dal PreCallAPI)* "Buongiorno Lorenzo. Il suo Galaxy S26 è in consegna con **GLS**: è in carico oggi e la consegna è prevista **domani entro le 18**. Posso aiutarla con altro?" [Lorenzo] "Perfetto. E sul nuovo Galaxy S27 che mi avete chiamato ieri: vale la pena passare dal S26?" [AI] *(dal brief: differenze vs S26, vantaggi, promo)* "Rispetto al suo S26: chip più veloce, batteria maggiore e una fotocamera molto più performante in condizioni di scarsa luce; sette anni di aggiornamenti. Con la promo lancio per i possessori di S26, fino a

150 euro di bonus permuta, il passaggio è vantaggioso." **[Lorenzo]** "Va bene, mi prenoto la promo. Procedete." **[AI]** (*conferma e chiude*) "Perfetto Lorenzo, ho registrato il suo interesse per il Galaxy S27 con la promo di lancio. La ricontatteremo. Grazie e buona giornata!"

Dopo la chiamata — in chat:

"Lorenzo ha confermato la promo per il Galaxy 27. Registra l'ordine."

(*L'agente registra l'ordine/pre-ordine del S27 per Lorenzo con la promo lancio.*)

Script:

"E adesso il tocco finale. Lorenzo richiama — e ascoltate: la Console sa **chi** è, sa **cosa** ha comprato, e sa **dove** è la consegna: 'in carico oggi, domani entro le 18'. Ha incrociato il brief dell'Atto 4 per rispondere sul S27 — e ha chiuso il pre-ordine con la promo. La chiamata che apre il dialogo e la memoria che risponde: è questa l'architettura — chi parla usa la conoscenza, chi ricorda è il CRM."

 **Pubblico vede:** la chiamata in vivavoce con saluto personalizzato e stato ordine; a fine chiamata la timeline che si aggiorna; il comando di registrazione ordine in chat e l'ordine S27 sul profilo.

Atto 6 — Il cerchio si chiude: il CRM ricorda tutto (2 min)

 **Azione:** nella chat, digitare:

"Cosa sappiamo di Lorenzo?"

Script:

(*L'agente riassume: 2 ordini — S26 in consegna GLS + S27 pre-ordinato con promo — consenso marketing, canale telefono, timeline delle interazioni.*)

"Eccolo: il cliente che dieci minuti fa era una riga di database ora è una storia — due ordini, un consenso, una campagna, tre interazioni. E la prossima volta che chiamerà, la Console lo riconoscerà di nuovo. **Niente più 'chi parla?'. È questo il CRM che ricorda.**"

👁️ **Pubblico vede:** profilo Lorenzo con i due ordini, consenso, preferenza canale e timeline completa.

Chiusura (2 min) — Call to action

🎤 Script:

"Ricapitolando. La memoria c'era già: un cliente con un ordine in consegna. In chat, l'operatore ha creato la campagna telefonica — e il sistema non scatta da solo: ogni invio e ogni attivazione passano dalla conferma umana. Poi il telefono ha squillato davvero: una campagna vera, una voce vera, un consenso registrato. La stessa conoscenza è diventata un brief multicanale, instradato per canale preferito. E quando il cliente ha richiamato, il CRM lo ha riconosciuto: nome, ordine, consegna, offerta — e ha chiuso il pre-ordine. Crm-A Console: **il CRM che ascolta, ricorda e agisce**. Domande?"

🖥️ **Azione:** aprire le domande. Tenere pronti `DEMO-RUNBOOK.md` (dettagli tecnici, Note NLPearl, numeri verificati) e `NLPEARL-SERVICES-PROMPT.md` (il prompt con cui sono stati generati i due servizi di chiamata) per chi chiede come è costruito.

Raggiungibilità — Tailscale funnel (scelta: Tailscale)

La Console deve essere raggiungibile da NLPearl per i callback (PreCall, webhook call/lead). **Scelta: Tailscale funnel** — già integrato nell'immagine Docker (l'entrypoint si unisce al tailnet, pubblica la porta 3100 e imposta `CRM_A_CONSOLE_PUBLIC_URL` da solo). Niente componenti extra: serve solo un auth key.

1. Nella Tailscale admin: creare un **reusable auth key**; abilitare **funnel** per il nodo nelle ACL (esempio `"funnels": [{"dnsName": "crm-a-console", "ports": [3100]}]`).
2. In `.env`:

```
TAILSCALE_AUTHKEY=tskey-auth-...
TAILSCALE_HOSTNAME=crm-a-console
```


3. Riavviare: `docker compose up -d --force-recreate`.
4. Verificare nei log: `Tailscale funnel: https://crm-a-console.<tailnet>.ts.net`.
5. Verifica esterna (prep, non sul palco): `curl -s -o /dev/null -w '%{http_code}' https://crm-a-console.<tailnet>.ts.net/api/integrations` → 200.

Alternativa non scelta: **zrok** (`zrok share public 3100`) funziona, ma richiede client/registrazione esterni e l'export manuale di `CRM_A_CONSOLE_PUBLIC_URL`; Tailscale è già nel container.

Checklist pre-demo (una scorsa prima di salire)

- ☐ `pnpm build:crm-a-plugins + docker compose up -d --build --force-recreate`
- ☐ `.env`: `CRM_A_PHONE_WEBHOOK_SECRET`, `NLPEARL_ACCOUNT_ID`, `NLPEARL_SECRET_KEY`, `TAILSCALE_AUTHKEY` (+ `TAILSCALE_HOSTNAME`)
- ☐ Funnel attivo: `https://crm-a-console.<tailnet>.ts.net/api/integrations` → 200 (da rete esterna)
- ☐ Seed eseguito + **Lorenzo**: Phone = tuo numero (E.164), Canale preferito = **Telefono**, Opt-in = sì (via chat o UI)
- ☐ Pearl **inbound** creata e **attiva** (da `NLPEARL-SERVICES-PROMPT.md` o `/api/nlpearl/inbound`)
- ☐ Numero outbound **verificato** per l'account (quello della chat in Atto 1)
- ☐ Chiamata outbound **collaudata** prima della demo (mai la prima volta sul palco)
- ☐ Pearl residue di collaudo rimosse dalla dashboard NLPearl
- ☐ Canali Telegram/email (Atto 4) collaudati — o script pronto a narrarne i `failed: [...]`
- ☐ Telefono carico, in vivavoce, numeri (inbound/outbound) a portata di mano; hard refresh browser